A Formal Security Analysis of SUCI-Stage Reused Key Binding in the 5G-AKA Family*

Yongho Ko[1] and Ilsun You[1,2†]

[1] Department of CyberSecurity, Kookmin University, Seoul, South Korea
koyh0911@kookmin.ac.kr
[2] KMU Global Research Center for ICT Convergence Security, Kookmin University, Seoul, Korea
ilsunu@gmail.com

Abstract

The 5G Authentication and Key Agreement (5G-AKA) family protects subscriber privacy in the initial identification stage through the Subscription Concealed Identifier (SUCI). However, the cryptographic relationship between the SUCI stage and the subsequent AKA stage remains relatively loose in many designs, leaving open questions regarding the security benefits of tighter cross-phase binding. This paper investigates the security implications of reusing SUCI-stage key materials to bind later AKA computations in the 5G-AKA family. Rather than focusing on a single protocol instance, we study this binding strategy as a general design principle and analyze its impact on representative 5G-AKA-style constructions. Our analysis focuses on two main questions: whether reused key binding introduces explicit cross-phase consistency between the SUCI and AKA stages, and whether it mitigates replay-driven failure-message-based linkability. To support this comparison, we also check basic SUCI-stage consistency using correspondence queries. We develop formal models in ProVerif and compare a baseline 5G-AKA-style setting with a bound setting that incorporates SUCI-stage reused key binding. In particular, we use correspondence queries to analyze basic SUCI-stage consistency and explicit cross-phase binding, and we employ choice-based observational equivalence to evaluate replay-driven linkability between same-subscriber and different-subscriber replay cases. The results show that both models preserve basic SUCI-stage consistency, but only the bound model satisfies the explicit cross-phase binding query. In addition, the bound model yields observational equivalence in the replay-driven linkability experiment, whereas the baseline model does not. These findings indicate that SUCI-stage reused key binding is not merely an implementation optimization, but a meaningful security design principle that can transform a weakly coupled SUCI-to-AKA relation into an explicitly bound one and can eliminate replay-driven distinguishability in the modeled setting.

1 Introduction

The 5G Authentication and Key Agreement (5G-AKA) family provides subscriber authentication and anchor-key establishment in 5G systems [3]. A major privacy-oriented enhancement over earlier generations is the use of the Subscription Concealed Identifier (SUCI), which protects the permanent subscriber identity during the initial access procedure [1]. Unless otherwise stated, references to 3GPP specifications in this paper correspond to the fixed versions listed in the bibliography. Nevertheless, prior studies have shown that, despite this improvement, the

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 24, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).
†Corresponding author

5G-AKA family still exhibits privacy limitations under active attacks, including replay-driven distinguishability and linkability issues [8, 4, 9].

This paper studies *SUCI-stage reused key binding* as a design principle in the 5G-AKA family. Rather than focusing on a single protocol variant, we investigate what security advantages can be obtained when key materials established during the SUCI stage are reused to bind later AKA computations. Our main objective is to clarify how such binding affects explicit cross-phase consistency and resistance to replay-driven linkability attacks. Recent studies on bound 5G-AKA-style constructions motivate this question, but our focus is the security effect of the design principle itself rather than that of any single protocol proposal [10, 7].

1.1 Related Work

Prior studies have shown that, although SUCI improves identity protection, the 5G-AKA family still has privacy limitations under active attacks [8, 9]. Formal analyses have further shown that the security guarantees of 5G-AKA depend strongly on the exact threat model and on how protocol phases are connected [4]. More recent protocol designs and reassessments have explored stronger coupling between the concealment and authentication stages, including approaches that reuse key material derived during the SUCI stage to improve freshness handling and resistance to linkability-related attacks [10, 7].

1.2 Motivation

The main motivation of this work is that the SUCI and AKA stages are consecutive but often only loosely coupled from a cryptographic perspective [3, 2]. If key materials established during the SUCI stage are reused in subsequent AKA computations, the resulting cross-phase binding may provide security benefits beyond identity concealment alone. In particular, such binding may reduce replay-driven distinguishability by making later authentication depend explicitly on the current SUCI-originated context [8, 10, 7]. This motivates a formal investigation of whether SUCI-stage reused key binding can strengthen the 5G-AKA family against classical failure-message-based linkability patterns.

1.3 Contributions

The contributions of this paper are summarized as follows.

- We identify SUCI-stage reused key binding as a reusable cross-phase security design principle in the 5G-AKA family.
- We present a formal analysis framework for evaluating its security implications with respect to explicit cross-phase binding and replay-driven linkability resistance.
- We model replay-based linkability scenarios in ProVerif using choice-based observational equivalence and compare same-subscriber and different-subscriber cases under reused-key binding.
- We show that SUCI-stage reused key binding can convert a weakly coupled SUCI-to-AKA relation into an explicitly bound one at the event level and can eliminate replay-driven distinguishability in our formal model.

The remainder of this paper is organized as follows. Section 2 introduces the background and design rationale. Section 3 presents the system and threat models. Section 4 describes the formal verification methodology. Section 5 discusses the verification results and security implications. Finally, Section 6 concludes the paper.

2 Background and Design Rationale

2.1 Notation

Table 1 summarizes the main symbols used in this paper. The notation is intentionally kept abstract so that the same analytical framework can be applied to both the baseline 5G-AKA-style setting and the bound setting with SUCI-stage reused key binding.

Table 1: Main notation used in the analysis

Symbol	**Description**
UE, SN, HN	User equipment, serving network, and home network
SUPI	Permanent subscriber identifier
SUCI	Subscription Concealed Identifier transmitted during initial access
K	Long-term subscriber key shared between UE and HN
SQN	Sequence-related freshness state
sk_{UE}, pk_{UE}	UE ephemeral secret/public key used in SUCI generation
sk_{HN}, pk_{HN}	HN long-term secret/public key for SUCI recovery
c_0, c_1, c_2	Abstract SUCI components derived during concealment
RAND	Baseline AKA challenge material
Y	Fresh challenge-related public value used in the bound setting
DHK	Diffie–Hellman-derived key material used for cross-phase binding
CONC	Concealed freshness-related field included in authentication data
MAC	Message authentication code used for AKA verification
RES'	UE response used for challenge verification
K_{AUSF}, K_{SEAF}	Anchor keys derived during later authentication processing

2.2 Baseline 5G-AKA Overview

Figure 1 presents the baseline 5G-AKA-style procedure used as the reference flow in this paper. At a high level, the protocol consists of two consecutive stages. In the first stage, the UE conceals its permanent identity by generating and sending a SUCI. The HN then verifies the concealed identifier, recovers the subscriber identity, and retrieves the associated subscriber state. In the second stage, the HN generates authentication vectors, the UE verifies the received challenge and computes its response, and the network finally verifies the response and establishes the anchor key.

The purpose of Figure 1 is not to reproduce the full standard message specification. Instead, it provides a compact procedural view of the baseline 5G-AKA-style flow that serves as the reference point for our later formal comparison. In particular, it shows that the SUCI stage and the AKA stage are sequentially connected, while leaving open the question of how strongly they are connected from a cryptographic viewpoint in the baseline abstraction used in this paper.

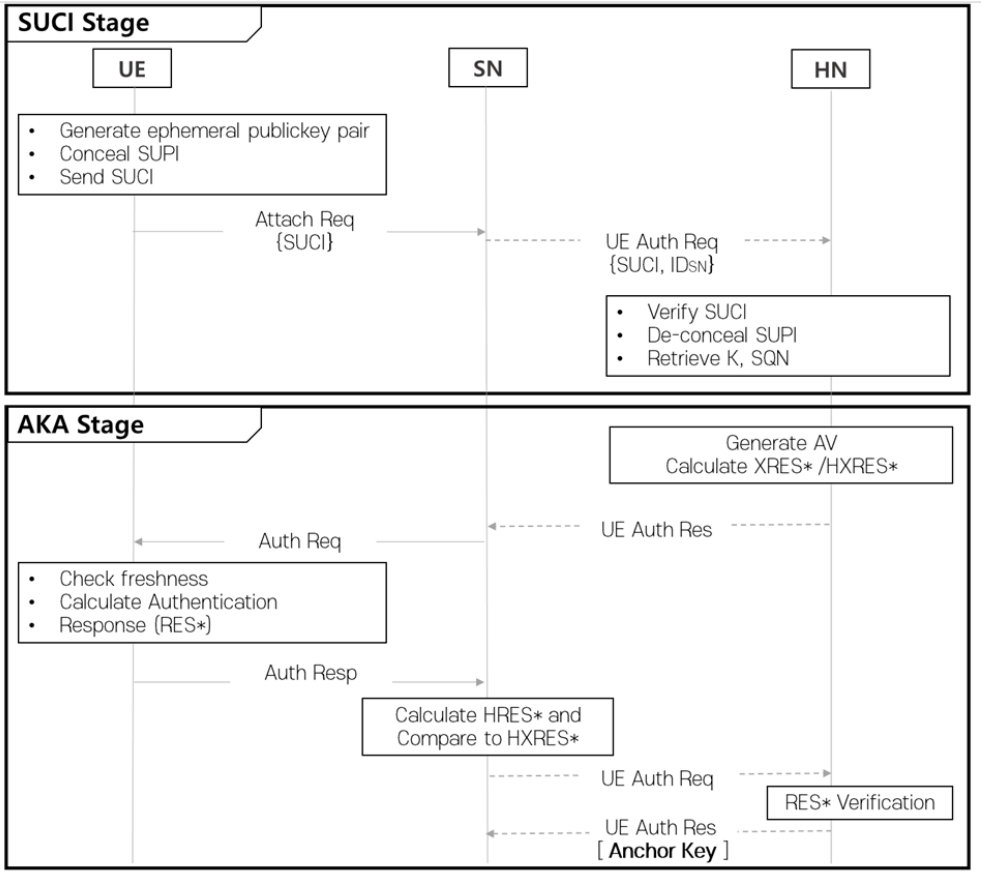


Figure 1: Simplified baseline 5G-AKA-style procedure used as the reference flow in this paper. The SUCI stage covers subscriber identity concealment and subscriber-state retrieval, while the AKA stage covers authentication-vector generation, UE response computation, RES* verification, and anchor-key establishment. In the baseline abstraction used in this paper, the later AKA verification is sequentially connected to, but not explicitly cryptographically bound to, the current SUCI-stage context.

2.3 Baseline Procedure

Algorithm 1 summarizes the baseline 5G-AKA-style procedure used in our analysis. It follows the same high-level flow as Figure 1, but describes the internal processing in a more abstract and verification-oriented form. In this baseline abstraction, the later AKA verification does not explicitly depend on the current SUCI-related context beyond the functional transition between the two phases.

2.4 Bound Procedure with SUCI-Stage Reused Key Binding

Algorithm 2 presents the bound setting analyzed in this paper. The overall message flow remains similar to the baseline reference flow shown in Figure 1 and Algorithm 1. However, the later

Algorithm 1: Baseline 5G-AKA-style processing

Input: $SUPI$, K, SQN, pk_{HN}
Output: RES', K_{AUSF}, K_{SEAF}, or failure
Generate a fresh UE ephemeral key pair (sk_{UE}, pk_{UE});
Compute SUCI components (c_0, c_1, c_2) using $(SUPI, sk_{UE}, pk_{HN})$;
Send SUCI $= (c_0, c_1, c_2)$ from UE to the network;
HN verifies the SUCI components and recovers $SUPI$;
HN retrieves the subscriber state (K, SQN) from its database;
HN generates fresh baseline challenge material $RAND$;
HN computes $CONC$ and MAC from $(RAND, K, SQN, AMF)$;
Send authentication data $(RAND, CONC, AMF, MAC)$ to UE;
UE verifies MAC using $(RAND, K)$;
if *MAC verification fails* **then**
  Abort with failure;
Recover the freshness-related value from $(CONC, RAND, K)$;
Compare the recovered value against the local SQN-related state;
if *freshness verification fails* **then**
  Abort with failure;
Compute RES' and the later key material;
Send RES' to the network;
SN/HN verify RES' and derive K_{AUSF} and K_{SEAF};

AKA computation now depends on key material that is cryptographically tied to the SUCI stage. Concretely, the HN introduces fresh challenge-related context and combines it with the UE-side SUCI context to derive bound authentication values. This makes later AKA acceptance explicitly dependent on the current SUCI-originated context in the bound abstraction.

Algorithm 2: Bound processing with SUCI-stage reused key binding

Input: $SUPI$, K, SQN, pk_{HN}
Output: RES', K_{AUSF}, K_{SEAF}, or failure
Generate a fresh UE ephemeral key pair (sk_{UE}, pk_{UE});
Compute SUCI components (c_0, c_1, c_2) using $(SUPI, sk_{UE}, pk_{HN})$;
Send SUCI $= (c_0, c_1, c_2)$ from UE to the network;
HN verifies the SUCI components and recovers $SUPI$;
HN retrieves the subscriber state (K, SQN) from its database;
HN generates fresh challenge-related context Y;
HN derives bound key material DHK from the current SUCI-related context and Y;
HN computes $CONC$ and MAC from (Y, DHK, K, SQN, AMF);
Send authentication data $(Y, CONC, AMF, MAC)$ to UE;
UE derives its local DHK from the current SUCI-related context and Y;
UE verifies MAC using (Y, DHK, K);
if *MAC verification fails* **then**
  Abort with failure;
Recover the freshness-related value from $(CONC, Y, DHK, K)$;
Compare the recovered value against the local SUCI-bound state;
if *freshness verification fails* **then**
  Abort with failure;
Compute RES' and the later key material using the bound context;
Send RES' to the network;
SN/HN verify RES' and derive K_{AUSF} and K_{SEAF};

2.5 Design Rationale and Security Scope

The key distinction between Algorithms 1 and 2 is not the overall message flow, but the source of dependency used in later AKA verification. In the baseline abstraction, the SUCI stage and the AKA stage are sequentially connected, yet the later authentication computation does not explicitly depend on the current SUCI-related context. In the bound setting, by contrast, the later AKA verification is cryptographically tied to the context established during SUCI generation.

This difference is security-relevant because replayed authentication material is processed differently in the two settings. If the later AKA stage does not explicitly depend on SUCI-stage context, stale authentication data may still induce subscriber-distinguishing behavior under replay. If, however, the later authentication computation is bound to current SUCI-stage context, replayed data are less likely to preserve the same observable behavior across different subscriber cases.

Our goal is not to claim that SUCI-stage reused key binding solves all privacy problems in the 5G-AKA family. Instead, we focus on a narrower but important question: what concrete security benefits can be obtained when SUCI-stage key materials are incorporated into later AKA computations in a cryptographically meaningful way? The subsequent sections therefore analyze basic SUCI-stage consistency, explicit cross-phase binding, and replay-driven linkability resistance using formal verification.

3 System and Threat Model

3.1 System Model

We consider a 5G-AKA-style authentication setting involving three logical entities: UE, SN, and HN [3, 2]. The baseline reference flow is summarized in Figure 1, while the detailed baseline and bound procedures used in our analysis are given in Algorithms 1 and 2, respectively.

In both settings, the UE first transmits a SUCI to conceal its permanent subscriber identity during the initial access stage, and the network subsequently executes the AKA stage to authenticate the subscriber and derive anchor keys for later communication [1, 3, 2]. Our purpose is not to analyze a specific implementation, but to compare two abstract 5G-AKA-family constructions under a common framework so that the security effect of SUCI-stage reused key binding can be isolated.

The baseline abstraction captures a structure in which SUCI generation and AKA verification are functionally consecutive but cryptographically weakly coupled. By contrast, the bound setting preserves the same high-level protocol flow while making the later AKA computation explicitly depend on key material established during the SUCI stage. This distinction forms the basis of our formal comparison [10, 7].

3.2 Threat Model

We adopt an active attacker model in which the adversary has full control over the public communication channel between the UE and the serving side. The attacker can intercept, replay, reorder, and inject messages, and may attempt to deliver stale authentication material to a target UE in order to trigger distinguishable protocol behavior [8, 9]. This model is appropriate for analyzing replay-driven linkability and failure-message-based distinguishability in the 5G-AKA family.

At the same time, we assume that the long-term subscriber key shared between the UE and HN remains uncompromised during the target session. We also assume that the HN-side subscriber database is trusted and that legitimate backend processing follows the intended protocol logic, as is standard in formal analyses of 5G authentication [4, 3]. Our focus is therefore not on database compromise or insider abuse, but on whether an external active adversary can exploit insufficient cross-phase binding between the SUCI and AKA stages.

The main attack scenario considered in this paper is replay-driven linkability. The attacker first obtains authentication material generated in an earlier session and later replays it against a subsequent session. We compare two cases: one in which the replayed material is delivered to the same subscriber and another in which it is delivered to a different subscriber. If these two cases induce distinguishable observable behavior, replay-driven linkability arises. If they remain observationally equivalent, the protocol resists this class of attack in our model [8, 9].

3.3 Comparison Scope and Security Goals

Our comparison is limited to the security effects induced by SUCI-stage reused key binding. In particular, we ask whether such binding changes the processing of stale authentication material in a way that reduces subscriber-distinguishing behavior. We do not claim that reused key binding automatically resolves all privacy limitations in the 5G-AKA family. Instead, we focus on the following goals:

- **G1: Basic SUCI-stage consistency.** The HN-side SUCI-acceptance event should correspond to a prior UE-side SUCI transmission event.
- **G2: Explicit cross-phase binding.** Later AKA acceptance should remain explicitly tied to the same SUCI-originated context when reused key binding is applied.
- **G3: Resistance to replay-driven linkability.** An active attacker should not be able to distinguish whether replayed authentication material is processed by the same subscriber or by a different subscriber solely through externally observable protocol behavior.

These goals are consistent with prior observations that privacy weaknesses in 5G-AKA are closely related not only to identity concealment itself but also to how later authentication computations react to replayed or adversarially manipulated data [8, 9, 10, 7].

In the following section, we formalize these goals in ProVerif. Goals G1–G2 are evaluated using correspondence queries, whereas Goal G3 is evaluated using choice-based observational equivalence between same-subscriber and different-subscriber replay scenarios.

4 Formal Verification Methodology

4.1 Verification Strategy

To analyze the security implications of SUCI-stage reused key binding, we construct two formal models in ProVerif: a baseline 5G-AKA-style model and a bound model in which key materials established during the SUCI stage are reused in later AKA computations. Our objective is not to prove the security of a single concrete implementation, but to isolate the security effect of cross-phase key binding under a common verification framework.

The verification strategy consists of two complementary parts. First, we use correspondence queries to examine basic SUCI-stage consistency and explicit cross-phase dependency in the

two models. Second, we use choice-based observational equivalence to evaluate replay-driven linkability by comparing the case in which replayed authentication material is delivered to the same subscriber with the case in which it is delivered to a different subscriber. This is a natural use of equivalence reasoning, since privacy-oriented properties such as anonymity, unlinkability, and non-traceability are more appropriately expressed as equivalence properties than as simple reachability properties in ProVerif [6, 5].

4.2 Goal-to-Query Mapping

Table 2 summarizes how each security goal is represented in the verification framework.

Table 2: Mapping between security goals and ProVerif analysis mechanisms

Goal	**Funcyion**	**Verification meaning**
G1	Correspondence query	The HN-side SUCI-acceptance event must correspond to a prior UE-side SUCI transmission event
G2	injective Correspondence query	Later AKA acceptance must remain explicitly tied to the same SUCI-originated context at the event level
G3	Observational equivalence	Same-subscriber replay and different-subscriber replay should be observationally indistinguishable

4.3 How Correspondence Queries Are Modeled

For Goals G1 and G2, we use correspondence queries. In ProVerif, this option is suitable when the property of interest can be expressed as a dependency between protocol events, that is, when a later event should imply the occurrence of an earlier event. In our models, we instrument the code with events representing key transitions such as SUCI transmission, successful SUCI recovery, authentication-data generation, and UE-side authentication acceptance.

This means that correspondence queries are not introduced merely as abstract labels. Rather, they are attached to the concrete control-flow points that matter for the analysis. Goal G1 is modeled by checking whether HN-side SUCI acceptance corresponds to a prior UE-side SUCI transmission. Goal G2 is modeled by checking whether later AKA acceptance remains explicitly tied to the same SUCI-originated context. In the baseline abstraction, this event-level relation is expected to be weak because the later AKA stage does not explicitly carry current SUCI-stage context. In the bound model, by contrast, the relevant events are parameterized with SUCI-related context, which makes the cross-phase dependency explicit in the verification code.

4.4 How `choice` Is Modeled

For Goal G3, we use the ProVerif `choice` construct inside a biprocess. This option is appropriate when the question is not whether a value can be reached, but whether two related executions are distinguishable by an attacker. In our setting, the two executions correspond to the same-subscriber replay case and the different-subscriber replay case.

From a modeling perspective, replay-driven linkability is encoded by keeping the replayed authentication tuple common across both sides of the biprocess while varying only the subscriber identity and the corresponding subscriber-specific state. Concretely, the left-hand side models

the case in which replayed authentication material is delivered to the same subscriber as in the original session, whereas the right-hand side models the case in which the same material is delivered to a different subscriber.

This means that `choice` is not used to represent nondeterministic protocol execution. Instead, it is used to encode two related executions within a single biprocess so that ProVerif can test whether an attacker can distinguish them observationally [6, 5]. In our implementation, the replayed authentication tuple is kept fixed, while the subscriber identity, long-term key, and associated state are varied through `choice`. This allows the verification code to isolate whether replayed authentication material induces subscriber-distinguishing behavior.

Algorithm 3: Choice-based modeling of replay-driven linkability

Input: Replayed authentication tuple T_{old}, subscriber states $(SUPI_A, K_A, SQN_A)$ and $(SUPI_B, K_B, SQN_B)$
Output: Observational-equivalence result
Fix the replayed authentication tuple T_{old} on both sides of the biprocess;
Set the left-hand side to the same-subscriber replay case;
Set the right-hand side to the different-subscriber replay case;
Instantiate the second-session UE process as follows:;
  UE_{replay}(;
    `choice`$[SUPI_A, SUPI_B]$,;
    `choice`$[K_A, K_B]$,;
    `choice`$[SQN_A, SQN_B], \ldots)$;
Run ProVerif observational-equivalence analysis;
if *equivalence is proved* **then**
  Conclude that the two replay cases are indistinguishable in the modeled scenario;
else
  Conclude that replay may induce subscriber-dependent observable behavior;

4.5 Interpretation of ProVerif Outputs

The two types of ProVerif output are interpreted differently. For correspondence queries, the result indicates whether the modeled event dependency holds under the stated assumptions. In our context, these results are used to support Goals G1 and G2. For observational equivalence, however, the ProVerif manual makes clear that failure is not reported as `false`. When ProVerif fails to establish equivalence, it returns `cannot be proved`, because the reconstructed trace only shows the first point at which the two sides may reduce differently and does not itself constitute a proof that observational equivalence is false [6].

Therefore, in our analysis, a result of `Observational equivalence is true` is interpreted as evidence supporting Goal G3 in the modeled replay scenario, whereas `Observational equivalence cannot be proved` is interpreted as an indication that replayed authentication material may induce distinguishable behavior and thus expose a linkability risk.

4.6 Result Presentation and Reproducibility

In the next section, we present the correspondence-query outputs for Goals G1 and G2 and the observational-equivalence outputs for Goal G3. Together, these results provide a structured comparison between the baseline model and the bound model.

To support transparency and reproducibility, the ProVerif source codes used in this study are publicly available in our GitHub repository.[1]

[1] https://github.com/yonghoko/ProVerif/tree/main/

5 Results and Discussion

5.1 Correspondence-Query Results for Goals G1–G2

Figure 2 presents the correspondence-query results for Goals G1 and G2 in a single side-by-side comparison. The left side corresponds to the baseline abstraction, whereas the right side corresponds to the bound model with SUCI-stage reused key binding. This combined presentation makes the difference between the two models directly visible at the event-correlation level.

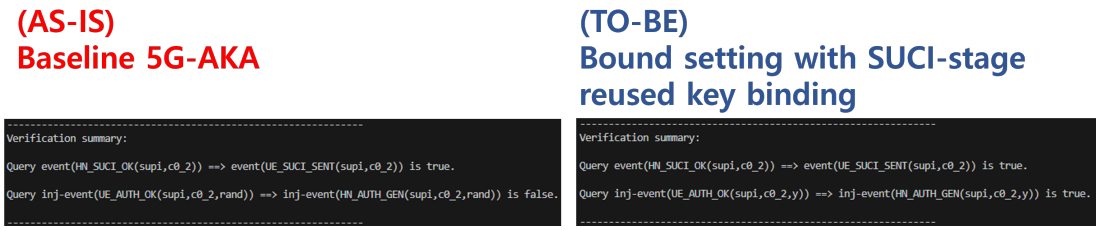


Figure 2: Combined ProVerif correspondence-query results for Goals G1 and G2. The left side shows the baseline abstraction, and the right side shows the bound model with SUCI-stage reused key binding. Both models satisfy the basic SUCI-stage consistency query (G1), whereas only the bound model satisfies the explicit cross-phase binding query (G2).

The figure shows that both models satisfy Goal G1, namely the basic SUCI-stage consistency property. In both cases, the HN-side SUCI-acceptance event corresponds to a prior UE-side SUCI transmission event. Thus, the baseline abstraction and the bound model both preserve the basic concealment-stage event relation.

By contrast, Goal G2 clearly distinguishes the two settings. In the baseline abstraction, the explicit cross-phase binding query is false, which indicates that later AKA acceptance is not explicitly tied to the same SUCI-originated context at the event level. In the bound model, however, the corresponding query is true. Accordingly, the combined correspondence-query results in Figure 2 provide direct formal evidence that SUCI-stage reused key binding transforms the SUCI-to-AKA relation from a weakly coupled one into an explicitly bound one.

5.2 Replay-Driven Linkability Result for Goal G3

Figure 3 presents the observational-equivalence results for Goal G3 in a single side-by-side comparison. The left side corresponds to the baseline abstraction, whereas the right side corresponds to the bound model with SUCI-stage reused key binding. As in Figure 2, this combined presentation is intended to make the difference between the two settings directly visible under the same replay-based verification scenario.

The figure shows that the baseline abstraction yields `Observational equivalence cannot be proved`, whereas the bound model yields `Observational equivalence is true`. This contrast is the central privacy-oriented result of the paper. In the baseline abstraction, replayed authentication material may still induce subscriber-dependent observable behavior, which means that same-subscriber replay and different-subscriber replay cannot be shown observationally equivalent in the modeled setting.

By contrast, the bound model satisfies the observational-equivalence check in the same replay scenario. Accordingly, Figure 3 provides direct formal evidence that SUCI-stage reused

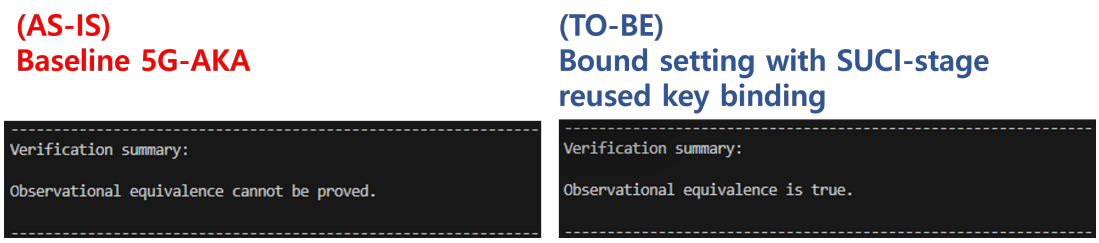


Figure 3: Combined ProVerif observational-equivalence results for Goal G3. The left side shows the baseline abstraction, and the right side shows the bound model with SUCI-stage reused key binding. The baseline model yields `Observational equivalence cannot be proved`, whereas the bound model yields `Observational equivalence is true`, indicating that replay-driven distinguishability remains in the baseline abstraction but disappears in the bound setting.

Table 3: Goal-oriented comparison between the baseline abstraction and the bound model with SUCI-stage reused key binding.

Goal	Baseline 5G-AKA	Bound setting	Interpretation
G1	True	True	Both models preserve basic concealment-stage consistency
G2	False	True	The bound model explicitly binds later AKA acceptance to the current SUCI-stage context
G3	Cannot be proved	True	The bound model eliminates replay-driven distinguishability in the modeled scenario

key binding eliminates replay-driven distinguishability in the compared setting. Taken together with the correspondence-query comparison in Figure 2, this result shows that the bound model not only introduces explicit event-level cross-phase binding, but also removes the observable replay difference that remains in the baseline abstraction.

5.3 Goal-Oriented Comparative Discussion

Table 3 summarizes the main differences between the baseline abstraction and the bound setting by aligning the verification results with Goals G1–G3. Read together, Figure 2, Figure 3, and Table 3 show a clear separation between the common property preserved by both models and the distinguishing properties introduced by SUCI-stage reused key binding.

Goal G1 is satisfied in both settings and therefore serves as a baseline consistency check. It confirms that both models preserve the basic SUCI-stage event relation. Goal G2, by contrast, distinguishes the two settings at the correspondence-query level: the baseline abstraction fails to establish explicit event-level cross-phase binding, whereas the bound model succeeds. Goal G3 then extends this distinction to the privacy-oriented setting: the baseline abstraction cannot

be shown observationally equivalent under replay, whereas the bound model can.

Taken together, these results show that the main formally demonstrated benefit of SUCI-stage reused key binding lies in two related improvements. First, it converts the SUCI-to-AKA relation from a weakly coupled one into an explicitly bound one at the event level. Second, it eliminates replay-driven distinguishability in the modeled linkability experiment. In this sense, the gain does not come from concealing more information in the abstract, but from restructuring how later authentication depends on the current SUCI-originated context.

5.4 Discussion

The results should be interpreted carefully. First, the observational-equivalence result for the bound model supports unlinkability only for the replay-based scenario captured by our abstraction. It should not be interpreted as a universal privacy guarantee for all possible attack surfaces in the 5G-AKA family. Second, the baseline result `cannot be proved` does not mean that ProVerif has returned `false`; rather, it indicates that the modeled executions may reduce differently and therefore cannot be shown observationally equivalent under the given setting. This interpretation follows the ProVerif manual and is consistent with our use of equivalence as a comparative privacy-analysis tool.

The correspondence-query results should also be interpreted within scope. Their role in this paper is not to establish a complete privacy claim by themselves, but to clarify how the SUCI-to-AKA dependency is represented in each model. In particular, the comparison in Figure 2 shows that the bound model makes the cross-phase relation explicit at the event level, whereas the baseline abstraction does not.

Within this scope, however, the combined comparison remains highly informative. The side-by-side difference in Figure 2 shows that SUCI-stage reused key binding introduces explicit cross-phase binding, and the side-by-side difference in Figure 3 shows that the same design choice eliminates one concrete replay-driven source of linkability in the modeled setting. Accordingly, the main contribution of this work is not the introduction of another protocol variant, but the formal clarification that stronger cryptographic dependency between SUCI generation and later AKA verification can yield measurable privacy benefits.

6 Conclusion

This paper examined the security implications of *SUCI-stage reused key binding* as a cross-phase design principle in the 5G-AKA family. Rather than focusing on a single protocol instance, we investigated whether reusing key materials established during the SUCI stage can provide meaningful security benefits when they are incorporated into subsequent AKA computations.

To answer this question, we developed a formal comparison framework based on ProVerif [6, 5]. We analyzed both a baseline 5G-AKA-style setting and a bound setting in which SUCI-stage key material is reused to strengthen the cryptographic dependency between the concealment stage and the authentication stage. The analysis covered basic SUCI-stage consistency, explicit cross-phase binding, and replay-driven linkability.

Our results show that the two ProVerif mechanisms play complementary roles. Correspondence-query results support Goal G1 by confirming that both models preserve basic SUCI-stage consistency, and they support Goal G2 by showing that only the bound model satisfies the explicit cross-phase binding query. Observational-equivalence results address Goal G3 directly: while the baseline setting yielded `Observational equivalence cannot be proved`, the bound setting yielded `Observational equivalence is true` in our replay-driven linkability experiment.

Taken together, these results indicate that SUCI-stage reused key binding can convert the SUCI-to-AKA relation from a weakly coupled one into an explicitly bound one in the model, and that this stronger binding eliminates replay-driven distinguishability in the compared setting.

The main implication of this work is that SUCI-stage key reuse should not be viewed merely as a mechanism for reusing previously available values. Its significance lies in whether it introduces meaningful cryptographic dependency between protocol phases that would otherwise remain only loosely connected. In this sense, SUCI-stage reused key binding can serve as a practical design principle for strengthening privacy-related security properties in the 5G-AKA family.

At the same time, our findings remain limited to the formal model considered in this paper. They do not imply that reused key binding automatically resolves all privacy limitations in the 5G-AKA family. Instead, they show that such binding can provide a measurable advantage against an important class of replay-driven linkability attacks. Future work may extend this analysis to broader privacy goals, richer threat models, and additional 5G-AKA variants.

Acknowledgments

This work was supported by Institute for Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2024-00397469, Development of Private 5G Security Technology for Integrated Private 5G and Enterprise Network Security, 100%).

References

[1] 3gpp ts 23.003: Numbering, addressing and identification. Tech. rep., 3GPP (2023), contains the definition of SUCI

[2] 3gpp ts 23.502: Procedures for the 5g system. Tech. rep., 3GPP (2025)

[3] 3gpp ts 33.501: Security architecture and procedures for 5g system. Tech. rep., 3GPP (2025), release 18, Version 18.9.0

[4] Basin, D., Dreier, J., Hirschi, L., Radomirović, S., Sasse, R., Stettler, V.: A formal analysis of 5g authentication. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (2018)

[5] Blanchet, B., Cheval, V., Smyth, B., Sylvestre, M.: Proverif with lemmas, induction, fast subsumption, and much more. In: IEEE Symposium on Security and Privacy (2022)

[6] Blanchet, B., Smyth, B., Cheval, V., Sylvestre, M.: ProVerif 2.05: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial (2023), available online

[7] Ko, Y., et al.: Formal security reassessment of the 5g-aka-fs protocol: Methodological corrections and augmented verification techniques. Sensors **24**(24), 7979 (2024)

[8] Koutsos, A.: The 5g-aka authentication protocol privacy. ACM Transactions on Privacy and Security (2019)

[9] Wang, Y., Zhang, Z., Xie, Y.: Privacy-preserving and standard-compatible aka protocol for 5g. In: USENIX Security Symposium (2021)

[10] You, I., Kim, G., Shin, S., Kwon, H., Kim, J., Baek, J.: 5g-aka-fs: A 5g authentication and key agreement protocol for forward secrecy. Sensors **24**(1), 159 (2024)